

Page Replacement

```
#include <stdio.h>

#define MAX 100

// Function to print frames
void printFrames(int frames[], int num_frames) {
    for (int i = 0; i < num_frames; i++) {
        if (frames[i] == -1)
            printf("- ");
        else
            printf("%d ", frames[i]);
    }
    printf("\n");
}

// ----- FIFO -----
void FIFO(int pages[], int n, int num_frames) {
    int frames[MAX], page_faults = 0;
    int next = 0;

    for (int i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\nFIFO Page Replacement:\n");

    for (int i = 0; i < n; i++) {
        int found = 0;

        // Check if page exists
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
    }
}
```

```

// Page Fault
if (!found) {
    frames[next] = pages[i];
    next = (next + 1) % num_frames;
    page_faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);
}

printf("Total Page Faults = %d\n", page_faults);
}

```

```

// ----- OPTIMAL -----
void Optimal(int pages[], int n, int num_frames) {
    int frames[MAX], page_faults = 0;

    for (int i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\nOptimal Page Replacement:\n");

    for (int i = 0; i < n; i++) {
        int found = 0;

        // Check if page exists
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
    }
}

```

```

// Page Fault
if (!found) {
    int empty = -1;

    // Check empty frame
    for (int j = 0; j < num_frames; j++) {
        if (frames[j] == -1) {
            empty = j;
            break;
        }
    }
}

```

```

    }
}

if (empty != -1) {
    frames[empty] = pages[i];
} else {
    int farthest = -1, pos = -1;

    for (int j = 0; j < num_frames; j++) {
        int k;

        for (k = i + 1; k < n; k++) {
            if (frames[j] == pages[k])
                break;
        }

        // If page not used again
        if (k == n) {
            pos = j;
            break;
        }

        // Find farthest used page
        if (k > farthest) {
            farthest = k;
            pos = j;
        }
    }

    frames[pos] = pages[i];
}

page_faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);
}

printf("Total Page Faults = %d\n", page_faults);
}

// ----- LRU -----
void LRU(int pages[], int n, int num_frames) {

```

```

int frames[MAX], time[MAX];
int page_faults = 0, counter = 0;

for (int i = 0; i < num_frames; i++) {
    frames[i] = -1;
    time[i] = 0;
}

printf("\nLRU Page Replacement:\n");

for (int i = 0; i < n; i++) {
    int found = 0;

    // Check if page exists
    for (int j = 0; j < num_frames; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            counter++;
            time[j] = counter;
            break;
        }
    }

    // Page Fault
    if (!found) {
        int empty = -1;

        // Check empty frame
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == -1) {
                empty = j;
                break;
            }
        }

        if (empty != -1) {
            frames[empty] = pages[i];
            counter++;
            time[empty] = counter;
        } else {
            // Find least recently used page
            int lru = time[0], pos = 0;

            for (int j = 1; j < num_frames; j++) {

```

```

        if (time[j] < lru) {
            lru = time[j];
            pos = j;
        }
    }

    frames[pos] = pages[i];
    counter++;
    time[pos] = counter;
}

page_faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);
}

printf("Total Page Faults = %d\n", page_faults);
}

// ----- MAIN -----
int main() {
    int pages[MAX], n, num_frames;

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);

    printf("Enter reference string:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &num_frames);

    FIFO(pages, n, num_frames);
    Optimal(pages, n, num_frames);
    LRU(pages, n, num_frames);

    return 0;
}

```

```

#include <stdio.h>

#define MAX 100

// Function to print frames
void printFrames(int frames[], int num_frames) {
    for (int i = 0; i < num_frames; i++) {
        if (frames[i] == -1)
            printf("- ");
        else
            printf("%d ", frames[i]);
    }
    printf("\n");
}

// ----- FIFO -----
void FIFO(int pages[], int n, int num_frames) {
    int frames[MAX], page_faults = 0;
    int next = 0;

    for (int i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\nFIFO Page Replacement:\n");

    for (int i = 0; i < n; i++) {
        int found = 0;

        // Check if page exists
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        // Page Fault
        if (!found) {
            frames[next] = pages[i];
            next = (next + 1) % num_frames;
            page_faults++;
        }

        printf("Page %d -> ", pages[i]);
    }
}

```

```

    printFrames(frames, num_frames);
}

printf("Total Page Faults = %d\n", page_faults);
}

// ----- OPTIMAL -----
void Optimal(int pages[], int n, int num_frames) {
    int frames[MAX], page_faults = 0;

    for (int i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\nOptimal Page Replacement:\n");

    for (int i = 0; i < n; i++) {
        int found = 0;

        // Check if page exists
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        // Page Fault
        if (!found) {
            int empty = -1;

            // Check empty frame
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    empty = j;
                    break;
                }
            }

            if (empty != -1) {
                frames[empty] = pages[i];
            } else {
                int farthest = -1, pos = -1;

                for (int j = 0; j < num_frames; j++) {

```

```

        int k;

        for (k = i + 1; k < n; k++) {
            if (frames[j] == pages[k])
                break;
        }

        // If page not used again
        if (k == n) {
            pos = j;
            break;
        }

        // Find farthest used page
        if (k > farthest) {
            farthest = k;
            pos = j;
        }
    }

    frames[pos] = pages[i];
}

page_faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);
}

printf("Total Page Faults = %d\n", page_faults);
}

// ----- LRU -----
void LRU(int pages[], int n, int num_frames) {
    int frames[MAX], time[MAX];
    int page_faults = 0, counter = 0;

    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\nLRU Page Replacement:\n");

```

```

for (int i = 0; i < n; i++) {
    int found = 0;

    // Check if page exists
    for (int j = 0; j < num_frames; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            counter++;
            time[j] = counter;
            break;
        }
    }

    // Page Fault
    if (!found) {
        int empty = -1;

        // Check empty frame
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == -1) {
                empty = j;
                break;
            }
        }

        if (empty != -1) {
            frames[empty] = pages[i];
            counter++;
            time[empty] = counter;
        } else {
            // Find least recently used page
            int lru = time[0], pos = 0;

            for (int j = 1; j < num_frames; j++) {
                if (time[j] < lru) {
                    lru = time[j];
                    pos = j;
                }
            }

            frames[pos] = pages[i];
            counter++;
            time[pos] = counter;
        }
    }
}

```

```

    }

    page_faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);
}

printf("Total Page Faults = %d\n", page_faults);
}

// ----- MAIN -----
int main() {
    int pages[MAX], n, num_frames;

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);

    printf("Enter reference string:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &num_frames);

    FIFO(pages, n, num_frames);
    Optimal(pages, n, num_frames);
    LRU(pages, n, num_frames);
    printf("\nName: Rishuraj Singh\n USN:1WA24CS232")

    return 0;
}

```

Enter number of pages in reference string: 21

Enter reference string:

5

0

1

0

2

3

0

2

4

3

3

2

0

2

1

2

7

0

1

1

0

Enter number of frames: 3

FIFO Page Replacement:

Page 5 -> 5 - -

Page 0 -> 5 0 -

Page 1 -> 5 0 1

Page 0 -> 5 0 1

Page 2 -> 2 0 1

Page 3 -> 2 3 1

Page 0 -> 2 3 0

Page 0 -> 4 2 0

Page 2 -> 4 2 0

Page 1 -> 4 2 1

Page 2 -> 4 2 1

Page 7 -> 7 2 1

Page 0 -> 7 0 1

Page 1 -> 7 0 1

Page 1 -> 7 0 1

Page 0 -> 7 0 1

Total Page Faults = 11

Optimal Page Replacement:

Page 5 -> 5 - -

Page 0 -> 5 0 -

Page 1 -> 5 0 1

Page 0 -> 5 0 1

Page 2 -> 2 0 1

Page 3 -> 2 0 3

Page 0 -> 2 0 3

Page 2 -> 2 0 3

Page 4 -> 2 4 3

Page 3 -> 2 4 3

Page 3 -> 2 4 3

Page 2 -> 2 4 3

Page 0 -> 2 0 3

Page 2 -> 2 0 3

Page 1 -> 2 0 1

Page 2 -> 2 0 1

Page 7 -> 7 0 1

Page 0 -> 7 0 1

Page 1 -> 7 0 1

Page 1 -> 7 0 1

Page 0 -> 7 0 1

```
Page 7 -> 7 0 1
Page 0 -> 7 0 1
Page 1 -> 7 0 1
Page 1 -> 7 0 1
Page 0 -> 7 0 1
Total Page Faults = 9
```

LRU Page Replacement:

```
Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 0 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 4 -> 2 0 4
Page 3 -> 2 3 4
Page 3 -> 2 3 4
Page 2 -> 2 3 4
Page 0 -> 2 3 0
Page 2 -> 2 3 0
Page 1 -> 2 1 0
Page 2 -> 2 1 0
Page 7 -> 2 1 7
Page 0 -> 2 0 7
Page 1 -> 1 0 7
Page 1 -> 1 0 7
Page 0 -> 1 0 7
Total Page Faults = 12
```

```
=== Code Execution Successful ===
```